Contents

1. Boilerplate II
   1.1. Imports II
   1.2. Type Definitions II
   1.3. Function Declarations III
2. String III
3. The Ast V
4. Lexer XVII
5. Parser XXIV
6. Pretty Printing XXV
7. Main XXIX

1. Boilerplate

1.1. Imports

```c
#include <ctype.h>
#include <stdint.h>
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
```

1.2. Type Definitions

All types are defined here, to allow for mutual recursion between them.

String is our own implementation of better strings than the C language.

```c
typedef struct String String;
```

Tokens are the atomic units of syntax that our lexer outputs.

```c
typedef enum TokenType TokenType;
typedef struct Token Token;
typedef struct TokenStack TokenStack;
```

The parser then parses them into an AST.

```c
typedef struct Ast Ast;
typedef enum TypeType TypeType;
typedef struct Type Type;
typedef struct TermField TermField;
typedef struct TypeField TypeField;
typedef enum NodeType NodeType;
typedef enum BinOpType BinOpType;

typedef struct HashMapEntry HashMapEntry;
typedef struct HashMap HashMap;
typedef struct Program Program;
```

This is for the parser.

```c
typedef struct Parser Parser;
```

1.3. Function Declarations

Next we declare all functions defined in the entire program before actually implementing them for the same reason. Function names typically follow the scheme of `<type>_<action>_<modifier>` and take as their first argument a value or a pointer of `<type>`.

Starting with the functions working on a String. These will be defined in Section 2.

```
void string_init(String* s, const char* initial);
String string_empty();
const char* string_to_cstr(const String* s);
String string_from_cstr(const char* initial);
void string_append(String* s, const char* str);
void string_push_char(String* s, char c);
void string_free(String* s);
String read_file(String filename);
```

The pretty printer.

```
void print_token(Token t);
```

2. String

A string is a pointer to a section of memory holding all the characters it contains. The string also has a length and a capacity, meaning our strings are not null-terminated. Capacity is the total space available for the character data. Length is the actual length of the string, both can but do not need be equal. The capacity is increased, whenever an operation is performed on the string that leads to its lenght becoming larger than the capacity. Since changing the capacity means allocating or freeing memory, the String implementation tries to minimize the amount of changes to capacity.

```
struct String
    {
    char* data;
    size_t length;
    size_t capacity;
    };
```

The members of String should not be set manually. Instead rely on the two constructor functions for initialising with a know sequence of characters or for creating an empty string. Of course, what is built must fall, therefore there is also a destructor.

```
void string_init(String* s, const char* initial)
    {
    size_t len = strlen(initial);
    s->length = len;
```

```
    s->capacity = len;
    s->data = (char*)malloc(s->capacity);
    strcpy(s->data, initial);
    }

String string_from_cstr(const char* initial)
    {
    String s;
    string_init(&s, initial);
    return s;
    }

const char* string_to_cstr(const String* s) { return s->data; }

String string_empty()
    {
    String s;
    s.data = (char*)malloc(1);
    s.data[0] = '\0';
    s.length = 0;
    s.capacity = 1;
    return s;
    }

void string_free(String* s)
    {
    free(s->data);
    s->data = NULL;
    s->length = 0;
    s->capacity = 0;
    }
```

Next we have some pretty obvious functions for working with strings.

```
void string_append(String* s, const char* str)
    {
    size_t new_len = s->length + strlen(str);

    if (new_len >= s->capacity)
        {
        s->capacity = new_len + 16;
        s->data = (char*)realloc(s->data, s->capacity);
        }

    strcat(s->data, str);
    s->length = new_len;
    }

void string_push_char(String* s, char c)
    {
    if (s->length + 1 >= s->capacity)
        {
        s->capacity = s->capacity * 2 + 16;
```

```
        s->data = (char*)realloc(s->data, s->capacity);
        }

    s->data[s->length] = c;
    s->length++;
    s->data[s->length] = '\0';
    }
```

And finally input and output with strings.

Add a function to write a string to file, maybe called write_to_file

```
String read_file(String filename)
    {
    String s = string_empty();
    const char* cstr = string_to_cstr(&filename);
    FILE* f = fopen(cstr, "r");

    if (f)
        {
        char buffer[1024];
        while (fgets(buffer, sizeof(buffer), f))
            {
            string_append(&s, buffer);
            }
        fclose(f);
        }
    return s;
    }
```

3. The Ast

The list of all possible kinds a type can be of.

```
enum TypeType
    {
    T_I8,
    T_I16,
    T_I32,
    T_I64,
    T_U8,
    T_U16,
    T_U32,
    T_U64,
    T_F16,
    T_F32,
    T_F64,

    T_Struct,
    T_Union,
    T_Enum,
```

```
    T_Pointer
    };
```

A type is a tag, which contains the information about its kind, and if the kind requires so the fiedl data, which is a union of all possible kinds of data, that can be associated with a type. Only one of these anonymous struct fields can be initlized for a concrete type. This design pattern, which is equivalent to an Algebraic Datatype (ADT) in languages like Haskell, OCaml or Rust, is used for all elements of the AST.

Types themselves are pretty trivial in Sonnet, the type language is strictly second-class and inferior to the term-language. There are primitves for numbers, both signed and unsigned, of various common bit lenghts. There are also composite types for pointers, and structs, unions and enums.

```
struct TypeField
    {

    String name;
    Type* type;
    };
struct Type
    {
    TypeType tag;
        union {
        struct
            {
            Type* pointing_to;
            } pointer_type;
        struct
            {
            struct TypeField* fields;
            size_t field_count;
            } struct_type;
        struct
            {
            struct TypeField* fields;
            size_t field_count;
            } union_type;
        struct
            {
            String* name;
            size_t field_count;
            } enum_type;

        } data;
    };

Type* new_type(Type type)
    {
    Type* ptr = malloc(sizeof(Type));
    if (ptr)
        *ptr = type;
    return ptr;
```

```
    }

Type* mk_type_prim(TypeType tag) { return new_type((Type){.tag = tag}); }

Type* mk_type_pointer(Type* pointing_to)
    {
    Type* t = new_type((Type){.tag = T_Pointer});
    t->data.pointer_type.pointing_to = pointing_to;
    return t;
    }
```

```
enum BinOpType
    {
    ADD,
    SUB,
    DIV,
    MUL,
    LessThan,
    LessThanEqual,
    MoreThan,
    MoreThanEqual,
    Equal,
    NotEqual
    };
```

```
enum NodeType
    {
    Var,
    BinOp,
    Fun,
    App,
    Struct,
    Union,
    Enum,
    Compound,
    VarDecl,
    VarSet,
    Loop,
    Ref,
    Deref,
    SetPtr,
    AccessPtr,
    I32Lit,
    StringLit
    };
```

```
struct TermField
    {
    String name;
```

```
    Ast* val;
    };

struct Ast
    {
    NodeType type;
        union {
        struct
            {
            String name;
            } var;
        struct
            {
            BinOpType op;
            Ast *left, *right;
            } bin_op;
        struct
            {
            TypeField* params;
            size_t param_count;
            Type* return_type;
            Ast* body;
            } fun;
        struct
            {
            Ast* callee;
            TermField* args;
            size_t arg_count;
            } app;
        struct
            {
            TermField* fields;
            size_t field_count;
            } struct_val;
        struct
            {
            TermField* fields;
            size_t field_count;
            } union_val;
        struct
            {
            String* name;
            size_t field_count;
            } enum_val;
        struct
            {
            Ast** exprs;
            size_t expr_count;
            } compound;
        struct
            {
            String name;
            Type* annot;
            Ast* init_expr;
            } var_decl;
        struct
```

```
            {
            String name;
            Ast* expr;
            } var_set;
        struct
            {
            Ast* expr;
            } loop;
        struct
            {
            Ast* expr;
            } ref;
        struct
            {
            Ast* expr;
            } deref;
        struct
            {
            Ast* target;
            Ast* value;
            } set_ptr;
        struct
            {
            Ast* target;
            String member;
            } access_ptr;
        struct
            {
            int32_t value;
            } i32_lit;
        struct
            {
            String value;
            } string_lit;
        } data;
    };
```

```
Ast* new_node(Ast ast)
    {
    Ast* ptr = malloc(sizeof(Ast));
    if (ptr)
        *ptr = ast;
    return ptr;
    }
```

There are helpr functions to make constructing an AST easier. These all start with mk_ followed by the name from the field of the anonymous union inside Ast.

```
Ast* mk_var(String name)
    {
    Ast* n = new_node((Ast){.type = Var});
```

```
    n->data.var.name = name;
    return n;
    }

Ast* mk_varc(const char* name)
    {
    Ast* n = new_node((Ast){.type = Var});
    n->data.var.name = string_from_cstr(name);
    return n;
    }

Ast* mk_bin_op(BinOpType op, Ast* left, Ast* right)
    {
    Ast* n = new_node((Ast){.type = BinOp});
    n->data.bin_op.op = op;
    n->data.bin_op.left = left;
    n->data.bin_op.right = right;
    return n;
    }

Ast* mk_fun(TypeField* params, size_t param_count, Type* return_type, Ast*
body)
    {
    Ast* n = new_node((Ast){.type = Fun});
    n->data.fun.params = params;
    n->data.fun.param_count = param_count;
    n->data.fun.return_type = return_type;
    n->data.fun.body = body;
    return n;
    }

Ast* mk_app(Ast* callee, TermField* args, size_t arg_count)
    {
    Ast* n = new_node((Ast){.type = App});
    n->data.app.callee = callee;
    n->data.app.args = args;
    n->data.app.arg_count = arg_count;
    return n;
    }

Ast* mk_struct_val(TermField* fields, size_t field_count)
    {
    Ast* n = new_node((Ast){.type = Struct});
    n->data.struct_val.fields = fields;
    n->data.struct_val.field_count = field_count;
    return n;
    }

Ast* mk_union_val(TermField* fields, size_t field_count)
    {
    Ast* n = new_node((Ast){.type = Union});
    n->data.union_val.fields = fields;
    n->data.union_val.field_count = field_count;
    return n;
    }
```

```
Ast* mk_enum_val(String* name, size_t field_count)
    {
    Ast* n = new_node((Ast){.type = Enum});
    n->data.enum_val.name = name;
    n->data.enum_val.field_count = field_count;
    return n;
    }

Ast* mk_compound(Ast** exprs, size_t expr_count)
    {
    Ast* n = new_node((Ast){.type = Compound});
    n->data.compound.exprs = exprs;
    n->data.compound.expr_count = expr_count;
    return n;
    }

Ast* mk_var_decl(String name, Type* annot, Ast* init_expr)
    {
    Ast* n = new_node((Ast){.type = VarDecl});
    n->data.var_decl.name = name;
    n->data.var_decl.annot = annot;
    n->data.var_decl.init_expr = init_expr;
    return n;
    }

Ast* mk_var_set(String name, Ast* expr)
    {
    Ast* n = new_node((Ast){.type = VarSet});
    n->data.var_set.name = name;
    n->data.var_set.expr = expr;
    return n;
    }

Ast* mk_loop(Ast* expr)
    {
    Ast* n = new_node((Ast){.type = Loop});
    n->data.loop.expr = expr;
    return n;
    }

Ast* mk_ref(Ast* expr)
    {
    Ast* n = new_node((Ast){.type = Ref});
    n->data.ref.expr = expr;
    return n;
    }

Ast* mk_deref(Ast* expr)
    {
    Ast* n = new_node((Ast){.type = Deref});
    n->data.deref.expr = expr;
    return n;
    }

Ast* mk_set_ptr(Ast* target, Ast* value)
    {
```

```
    Ast* n = new_node((Ast){.type = SetPtr});
    n->data.set_ptr.target = target;
    n->data.set_ptr.value = value;
    return n;
    }

Ast* mk_access_ptr(Ast* target, String member)
    {
    Ast* n = new_node((Ast){.type = AccessPtr});
    n->data.access_ptr.target = target;
    n->data.access_ptr.member = member;
    return n;
    }

Ast* mk_i32_lit(int32_t lit)
    {
    Ast* n = new_node((Ast){.type = I32Lit});
    n->data.i32_lit.value = lit;
    return n;
    }

Ast* mk_string_lit(String lit)
    {
    Ast* n = new_node((Ast){.type = I32Lit});
    n->data.string_lit.value = lit;
    return n;
    }
```

```
void type_free(Type* t)
    {
    if (!t)
        return;

    if (t->tag == T_Struct)
        {
        for (size_t i = 0; i < t->data.struct_type.field_count; i++)
            {
            string_free(&t->data.struct_type.fields[i].name);
            type_free(t->data.struct_type.fields[i].type);
            }
        free(t->data.struct_type.fields);
        }
    else if (t->tag == T_Union)
        {
        for (size_t i = 0; i < t->data.union_type.field_count; i++)
            {
            string_free(&t->data.union_type.fields[i].name);
            type_free(t->data.union_type.fields[i].type);
            }
        free(t->data.union_type.fields);
        }
    else if (t->tag == T_Enum)
        {
```

```
        for (size_t i = 0; i < t->data.enum_type.field_count; i++)
            {
            string_free(&t->data.enum_type.name[i]);
            }
        free(t->data.enum_type.name);
        }
    free(t);
    }

void ast_free(Ast* node)
    {
    if (!node)
        return;

    switch (node->type)
        {
    case Var:
        string_free(&node->data.var.name);
        break;

    case BinOp:
        ast_free(node->data.bin_op.left);
        ast_free(node->data.bin_op.right);
        break;

    case Fun:
        for (size_t i = 0; i < node->data.fun.param_count; i++)
            {
            string_free(&node->data.fun.params[i].name);
            type_free(node->data.fun.params[i].type);
            }
        free(node->data.fun.params);
        type_free(node->data.fun.return_type);
        ast_free(node->data.fun.body);
        break;

    case App:
        ast_free(node->data.app.callee);
        for (size_t i = 0; i < node->data.app.arg_count; i++)
            {
            string_free(&node->data.app.args[i].name);
            ast_free(node->data.app.args[i].val);
            }
        free(node->data.app.args);
        break;

    case Struct:
        for (size_t i = 0; i < node->data.struct_val.field_count; i++)
            {
            string_free(&node->data.struct_val.fields[i].name);
            ast_free(node->data.struct_val.fields[i].val);
            }
        free(node->data.struct_val.fields);
        break;

    case Union:
```

```
        for (size_t i = 0; i < node->data.union_val.field_count; i++)
            {
            string_free(&node->data.union_val.fields[i].name);
            ast_free(node->data.union_val.fields[i].val);
            }
        free(node->data.union_val.fields);
        break;

    case Enum:
        for (size_t i = 0; i < node->data.enum_val.field_count; i++)
            {
            string_free(&node->data.enum_val.name[i]);
            }
        free(node->data.enum_val.name);
        break;

    case Compound:
        for (size_t i = 0; i < node->data.compound.expr_count; i++)
            {
            ast_free(node->data.compound.exprs[i]);
            }
        free(node->data.compound.exprs);
        break;

    case VarDecl:
        string_free(&node->data.var_decl.name);
        type_free(node->data.var_decl.annot);
        ast_free(node->data.var_decl.init_expr);
        break;

    case VarSet:
        string_free(&node->data.var_set.name);
        ast_free(node->data.var_set.expr);
        break;

    case Loop:
        ast_free(node->data.loop.expr);
        break;

    case Ref:
        ast_free(node->data.ref.expr);
        break;

    case Deref:
        ast_free(node->data.deref.expr);
        break;

    case SetPtr:
        ast_free(node->data.set_ptr.target);
        ast_free(node->data.set_ptr.value);
        break;

    case AccessPtr:
        ast_free(node->data.access_ptr.target);
        string_free(&node->data.access_ptr.member);
        break;
```

```
    case I32Lit:
        break;

    case StringLit:
        string_free(&node→data.string_lit.value);
        break;
        }

    free(node);
    }
```

Since in a Sonnet program all term and type definitions come into being at the same time, unlike in C were you have to predeclare everything, we need to make a Program hold to different mpas, one linking names to types, for type definitions, and the other one mapping names to terms, for term definitions. Both are realised by HashMap. The HashMap stores values with the void*-type, instead of having to different implementations for types and terms at the cost of less type safety - but you only life once!

```
struct HashMapEntry
    {
    String key;
    void* value;
    struct HashMapEntry* next;
    };

struct HashMap
    {
    HashMapEntry** buckets;
    size_t capacity;
    size_t count;
    };
```

For computing the hash of a given name we use the djb2-hash-function.

```
unsigned long hash_string(const String* s)
    {
    unsigned long hash = 5381;
    for (size_t i = 0; i < s→length; i++)
        {
        hash = ((hash << 5) + hash) + s→data[i];
        }
    return hash;
    }
```

As previously discussed, members of structs should in most cases assumed to be private. This is also the case for HashMap, therefore use these functions for working with one. Using these we can initialize a hash map with a given number of buckets, insert a key-value pair into the hash map, and retrieve the value associated with a given key.

```
void hashmap_init(HashMap* map, size_t capacity)
    {
    map→capacity = capacity;
    map→count = 0;
    map→buckets = (HashMapEntry**)calloc(capacity, sizeof(HashMapEntry*));
    }

void hashmap_insert(HashMap* map, String key, void* value)
    {
    unsigned long hash = hash_string(&key) % map→capacity;

    HashMapEntry* entry = malloc(sizeof(HashMapEntry));
    entry→key = key;
    entry→value = value;

    entry→next = map→buckets[hash];
    map→buckets[hash] = entry;
    map→count++;
    }

void* hashmap_get(HashMap* map, String key)
    {
    unsigned long hash = hash_string(&key) % map→capacity;
    HashMapEntry* entry = map→buckets[hash];

    while (entry)
        {
        if (strcmp(entry→key.data, key.data) == 0)
            return entry→value;
        entry = entry→next;
        }
    return NULL;
    }
```

Since a hash map can store all kinds of data, this data must be freed to, when the hash map is destroyed. Therefore a destructor for the values can be given to the hashmap_free-function.

```
void hashmap_free(HashMap* map, void (*value_free_func)(void*))
    {
    if (!map || !map→buckets)
        return;

    for (size_t i = 0; i < map→capacity; i++)
        {
        HashMapEntry* entry = map→buckets[i];
        while (entry)
            {
            HashMapEntry* next = entry→next;

            string_free(&entry→key);

            if (value_free_func)
                {
```

```
                value_free_func(entry→value);
                }

            free(entry);
            entry = next;
            }
        }
    free(map→buckets);
    map→buckets = NULL;
    }
```

A program is just two maps for type and term synonyms. Were possible functions are defined to better suit the more specialized needs of a program compared to the general hash map.

```
struct Program
    {
    HashMap type_definitions;
    HashMap term_definitions;
    };

Program program_init(size_t capacity)
    {
    Program p;
    hashmap_init(&p.type_definitions, capacity);
    hashmap_init(&p.term_definitions, capacity);
    return p;
    }

void program_free(Program* p)
    {
    hashmap_free(&p→type_definitions, (void (*)(void*))type_free);
    hashmap_free(&p→term_definitions, (void (*)(void*))ast_free);
    }

void program_add_type(Program* p, String name, Type* type)
    {
    hashmap_insert(&p→type_definitions, name, (void*)type);
    }

void program_add_term(Program* p, String name, Ast* term)
    {
    hashmap_insert(&p→term_definitions, name, (void*)term);
    }
```

4. Lexer

```
enum TokenType
```

```
    {
    TOK_EOF,
    TOK_IDENT,
    TOK_INT,
    TOK_STRING,
    TOK_ASSIGN,
    TOK_LEFT_ARROW,
    TOK_RIGHT_ARROW,
    TOK_DOT,
    TOK_STAR,
    TOK_EXCLAMATION,
    TOK_LPAREN,
    TOK_RPAREN,
    TOK_LBRACE,
    TOK_RBRACE,
    TOK_COMMA,
    TOK_KW_FUN,
    TOK_KW_LET,
    TOK_KW_IF,
    TOK_KW_THEN,
    TOK_KW_ELSE,
    TOK_KW_DEFTYPE,
    TOK_KW_STRUCT,
    TOK_KW_ENUM,
    TOK_KW_UNION,
    TOK_PLUS,
    TOK_MINUS,
    TOK_LESS,
    TOK_LESS_EQ,
    TOK_MORE,
    TOK_MORE_EQ,
    TOK_EQ,
    TOK_NOT_EQ
    };

struct Token
    {
    TokenType type;
        union {
        String ident_or_lit;
        int32_t lit;
        } data;
    };

Token tok_empty(TokenType t)
    {
    Token tok;
    tok.type = t;
    return tok;
    }

struct TokenStack
    {
    Token* tokens;
    size_t top;
    size_t capacity;
```

```
    };

void stack_free(TokenStack* s)
    {
    if (!s || !s->tokens)
        return;

    for (size_t i = 0; i < s->top; i++)
        {
        Token* t = &s->tokens[i];

        if (t->type == TOK_IDENT || t->type == TOK_STRING)
            {
            string_free(&t->data.ident_or_lit);
            }
        }

    free(s->tokens);

    s->tokens = NULL;
    s->top = 0;
    s->capacity = 0;
    }

TokenStack stack_empty()
    {
    TokenStack s;
    s.tokens = NULL;
    s.top = 0;
    s.capacity = 0;
    return s;
    }

void stack_push(TokenStack* s, Token t)
    {
    if (s->top >= s->capacity)
        {
        s->capacity = s->capacity == 0 ? 16 : s->capacity * 2;
        s->tokens = realloc(s->tokens, sizeof(Token) * s->capacity);
        }
    s->tokens[s->top++] = t;
    }

Token stack_pop(TokenStack* s)
    {
    if (s->top == 0)
        return tok_empty(TOK_EOF);
    return s->tokens[--s->top];
    }

Token stack_peek(TokenStack* s)
    {
    if (s->top == 0)
        return tok_empty(TOK_EOF);
    return s->tokens[s->top - 1];
    }
```

```
void stack_reverse(TokenStack* s)
    {
    if (s->top == 0)
        return;

    TokenStack s2 = stack_empty();

    while (s->top != 0)
        {
        stack_push(&s2, stack_pop(s));
        }

    Token* temp_tokens = s->tokens;
    s->tokens = s2.tokens;
    s2.tokens = temp_tokens;

    size_t temp_top = s->top;
    s->top = s2.top;
    s2.top = temp_top;

    size_t temp_cap = s->capacity;
    s->capacity = s2.capacity;
    s2.capacity = temp_cap;

    free(s2.tokens);
    }

int is_ident_char(char c) { return isalnum(c) || c == '_'; }

TokenType get_keyword_type(const char* str)
    {
    if (strcmp(str, "fun") == 0)
        return TOK_KW_FUN;
    if (strcmp(str, "let") == 0)
        return TOK_KW_LET;
    if (strcmp(str, "if") == 0)
        return TOK_KW_IF;
    if (strcmp(str, "then") == 0)
        return TOK_KW_THEN;
    if (strcmp(str, "else") == 0)
        return TOK_KW_ELSE;
    if (strcmp(str, "deftype") == 0)
        return TOK_KW_DEFTYPE;
    if (strcmp(str, "struct") == 0)
        return TOK_KW_STRUCT;
    if (strcmp(str, "enum") == 0)
        return TOK_KW_ENUM;
    if (strcmp(str, "union") == 0)
        return TOK_KW_UNION;
    return TOK_IDENT;
    }

void lex(const String* input, TokenStack* stack)
    {
    size_t i = 0;
```

```
const char* data = input->data;
size_t len = input->length;

while (i < len)
    {
    char c = data[i];

    if (isspace(c))
        {
        i++;
        continue;
        }

    if (isalpha(c) || c == '_')
        {
        String ident = string_empty();
        while (i < len && is_ident_char(data[i]))
            {
            string_push_char(&ident, data[i]);
            i++;
            }

        TokenType type = get_keyword_type(ident.data);
        Token t = tok_empty(type);

        if (type == TOK_IDENT)
            {
            t.data.ident_or_lit = ident;
            }
        else
            {
            string_free(&ident);
            }

        stack_push(stack, t);
        continue;
        }

    if (isdigit(c))
        {
        int32_t val = 0;
        while (i < len && isdigit(data[i]))
            {
            val = val * 10 + (data[i] - '0');
            i++;
            }
        Token t = tok_empty(TOK_INT);
        t.data.lit = val;
        stack_push(stack, t);
        continue;
        }

    if (c == '"')
        {
        i++;
        String str_val = string_empty();
```

```
            while (i < len && data[i] ≠ '"')
                {
                string_push_char(&str_val, data[i]);
                i++;
                }

            if (i < len && data[i] == '"')
                {
                i++;
                }

            Token t = tok_empty(TOK_STRING);
            t.data.ident_or_lit = str_val;
            stack_push(stack, t);
            continue;
            }

        Token t;
        switch (c)
            {
        case '=':
            if (i + 1 < len && data[i + 1] == '=')
                {
                t.type = TOK_EQ;
                i += 2;
                }
            else
                {
                t.type = TOK_ASSIGN;
                i++;
                }
            stack_push(stack, t);
            break;
        case '<':
            if (i + 1 < len && data[i + 1] == '=')
                {
                t.type = TOK_LESS_EQ;
                i += 2;
                }
            else if (i + 1 < len && data[i + 1] == '-')
                {
                t.type = TOK_LEFT_ARROW;
                i += 2;
                }
            else
                {
                t.type = TOK_LESS;
                i++;
                }
            stack_push(stack, t);
            break;
        case '>':
            if (i + 1 < len && data[i + 1] == '=')
                {
                t.type = TOK_MORE_EQ;
```

```
                i += 2;
                }
            else
                {
                t.type = TOK_MORE;
                i++;
                }
            stack_push(stack, t);
            break;
        case '-':
            if (i + 1 < len && data[i + 1] == '>')
                {
                t.type = TOK_RIGHT_ARROW;
                i += 2;
                }
            else
                {
                t.type = TOK_MINUS;
                i++;
                }
            stack_push(stack, t);
            break;
        case '!':
            if (i + 1 < len && data[i + 1] == '=')
                {
                t.type = TOK_NOT_EQ;
                i += 2;
                }
            else
                {
                t.type = TOK_EXCLAMATION;
                i++;
                }
            stack_push(stack, t);
            break;
        case '+':
            t.type = TOK_PLUS;
            i++;
            stack_push(stack, t);
            break;
        case '.':
            t.type = TOK_DOT;
            i++;
            stack_push(stack, t);
            break;
        case '*':
            t.type = TOK_STAR;
            i++;
            stack_push(stack, t);
            break;
        case '(':
            t.type = TOK_LPAREN;
            i++;
            stack_push(stack, t);
            break;
        case ')':
```

```
            t.type = TOK_RPAREN;
            i++;
            stack_push(stack, t);
            break;
        case '{':
            t.type = TOK_LBRACE;
            i++;
            stack_push(stack, t);
            break;
        case '}':
            t.type = TOK_RBRACE;
            i++;
            stack_push(stack, t);
            break;
        case ',':
            t.type = TOK_COMMA;
            i++;
            stack_push(stack, t);
            break;
        default:
            i++;
            break;
            }
        }

    Token eof_tok;
    eof_tok.type = TOK_EOF;
    stack_push(stack, eof_tok);
    }
```

5. Parser

```
void expect_peek(TokenStack* stack, TokenType expected)
    {
    TokenType actual = stack_peek(stack).type;
    if (actual != expected)
        {
        fprintf(stderr, "Expected token type %d, but got %d\n", expected,
                actual);
        exit(EXIT_FAILURE);
        }
    }

void expect_consume(TokenStack* stack, TokenType expected)
    {
    TokenType actual = stack_pop(stack).type;
    if (actual != expected)
        {
        fprintf(stderr, "Expected token type %d, but got %d\n", expected,
                actual);
        exit(EXIT_FAILURE);
```

```
        }
    }

void parse(TokenStack* stack, Program* p)
    {
    expect_peek(stack, TOK_IDENT);
    String fun_name = stack_pop(stack).data.ident_or_lit;
    expect_consume(stack, TOK_ASSIGN);

    TypeField* params = malloc(sizeof(TypeField));
    params[0].name = string_from_cstr("x");
    params[0].type = mk_type_prim(T_I32);

    program_add_term(
        p, fun_name,
        mk_fun(params, 1, mk_type_prim(T_I32),
mk_var(string_from_cstr("x"))));
    }
```

6. Pretty Printing

We define prtty printing functions for every object to easily inspect for debugging purposes:

```
void print_indent(int depth)
    {
    for (int i = 0; i < depth; i++)
        printf("\t");
    }
```

Pretty Printing for the lexer and its tokens.

```
void print_token(Token t)
    {
    switch (t.type)
        {
    case TOK_EOF:
        printf("EOF");
        break;
    case TOK_IDENT:
        printf("IDENT(\"%s\")", t.data.ident_or_lit.data);
        break;
    case TOK_INT:
        printf("INT(%d)", t.data.lit);
        break;
    case TOK_STRING:
        printf("STRING(\"%s\")", t.data.ident_or_lit.data);
        break;
    case TOK_ASSIGN:
        printf("=");
```

```
        break;
    case TOK_LEFT_ARROW:
        printf("←");
        break;
    case TOK_RIGHT_ARROW:
        printf("→");
        break;
    case TOK_DOT:
        printf(".");
        break;
    case TOK_STAR:
        printf("*");
        break;
    case TOK_EXCLAMATION:
        printf("!");
        break;
    case TOK_LPAREN:
        printf("(");
        break;
    case TOK_RPAREN:
        printf(")");
        break;
    case TOK_LBRACE:
        printf("{");
        break;
    case TOK_RBRACE:
        printf("}");
        break;
    case TOK_COMMA:
        printf(",");
        break;
    case TOK_KW_FUN:
        printf("KW_FUN");
        break;
    case TOK_KW_LET:
        printf("KW_LET");
        break;
    case TOK_KW_IF:
        printf("KW_IF");
        break;
    case TOK_KW_THEN:
        printf("KW_THEN");
        break;
    case TOK_KW_ELSE:
        printf("KW_ELSE");
        break;
    case TOK_KW_DEFTYPE:
        printf("KW_DEFTYPE");
        break;
    case TOK_KW_STRUCT:
        printf("KW_STRUCT");
        break;
    case TOK_KW_ENUM:
        printf("KW_ENUM");
        break;
    case TOK_KW_UNION:
```

```
        printf("KW_UNION");
        break;
    case TOK_PLUS:
        printf("+");
        break;
    case TOK_MINUS:
        printf("-");
        break;
    case TOK_LESS:
        printf("<");
        break;
    case TOK_LESS_EQ:
        printf("⩽");
        break;
    case TOK_MORE:
        printf(">");
        break;
    case TOK_MORE_EQ:
        printf("⩾");
        break;
    case TOK_EQ:
        printf("==");
        break;
    case TOK_NOT_EQ:
        printf("≠");
        break;
        }
    printf("\n");
    }
```

Printing types.

```
void print_type(Type* t, int depth)
    {
    if (!t)
        return;
    print_indent(depth);

    switch (t→tag)
        {
    case T_I8:
        printf("I8\n");
        break;
    case T_I16:
        printf("I16\n");
        break;
    case T_I32:
        printf("I32\n");
        break;
    case T_I64:
        printf("I64\n");
        break;
    case T_U8:
```

```
        printf("U8\n");
        break;
    case T_U16:
        printf("U16\n");
        break;
    case T_U32:
        printf("U32\n");
        break;
    case T_U64:
        printf("U64\n");
        break;
    case T_F16:
        printf("F16\n");
        break;
    case T_F32:
        printf("F32\n");
        break;
    case T_F64:
        printf("F64\n");
        break;
    case T_Pointer:
        printf("*");
        print_type(t->data.pointer_type.pointing_to, depth);
        break;
    case T_Struct:
        printf("Struct:\n");
        for (size_t i = 0; i < t->data.struct_type.field_count; i++)
            {
            print_indent(depth + 1);
            printf("Field: %s\n", t->data.struct_type.fields[i].name.data);
            print_type(t->data.struct_type.fields[i].type, depth + 2);
            }
        break;
    case T_Union:
        printf("Union:\n");
        for (size_t i = 0; i < t->data.union_type.field_count; i++)
            {
            print_indent(depth + 1);
            printf("Field: %s\n", t->data.union_type.fields[i].name.data);
            print_type(t->data.union_type.fields[i].type, depth + 2);
            }
        break;
    case T_Enum:
        printf("Enum:\n");
        for (size_t i = 0; i < t->data.enum_type.field_count; i++)
            {
            print_indent(depth + 1);
            printf("Field: %s\n", t->data.enum_type.name[i].data);
            }
        break;
        }
    }
```

Printing the ast.

```
const char* binop_to_str(BinOpType op)
    {
    switch (op)
        {
    case ADD:
        return "+";
    case SUB:
        return "-";
    case DIV:
        return "/";
    case MUL:
        return "*";
    case LessThan:
        return "<";
    case LessThanEqual:
        return "≤";
    case MoreThan:
        return ">";
    case MoreThanEqual:
        return "≥";
    case Equal:
        return "==";
    case NotEqual:
        return "≠";
        }
    }
```

7. Main

```
int main(int argc, char** argv)
    {
    if (argc < 2)
        {
        printf("Usage: %s <filename>\n", argv[0]);
        return 1;
        }
    String filename = string_from_cstr(argv[1]);
    String source_code = read_file(filename);

    if (source_code.length == 0)
        {
        printf("Error: Could not read file '%s' or file is empty.\n",
argv[1]);
        string_free(&source_code);
        return 1;
        }

    TokenStack stack = stack_empty();
    lex(&source_code, &stack);
    string_free(&source_code);
```

```
    stack_reverse(&stack);

    Program p = program_init(1024);
    parse(&stack, &p);

    stack_free(&stack);

    return 0;
    }
```